# Semantic HTML

Writing HTML that means something

**About These Notes**

These notes are written for two levels of learners:

**Beginner:** New to HTML — learn what semantic elements exist and why they matter.

**Intermediate:** Already know basic HTML — learn how to use semantic elements correctly and accessibly.

Throughout these notes, look for the  and ⚡ icons to find content tailored to your level.

**Tools Used in This Module**

**Visual Studio Code (VSCode)** — write and preview all HTML files locally.

Install the Live Server extension: Extensions panel → search 'Live Server' → Install.

Right-click any .html file → Open with Live Server to get instant browser previews.

## 01 What Is Semantic HTML?

# What Is Semantic HTML?

## 1.1 HTML with Meaning

Every HTML element has a name — but not every element tells you what its content means. Semantic HTML means using the right element for the right job, so that the element name itself communicates the purpose of the content inside it.

```
example.html
<!-- Non-semantic: a div with styling, no meaning -->
<div class="big-title">Welcome to TS Academy</div>

<!-- Semantic: an <h1> tells everyone this is the main heading -->
<h1>Welcome to TS Academy</h1>
```

Both might look identical in a browser. But only the `<h1>` version communicates that this text is the primary heading of the page.

**Beginner: Think of it like this**

Imagine writing a letter. You could write everything in the same pen with no structure — or you could use a header at the top, paragraphs in the body, and a signature at the bottom.

Semantic HTML is like the structure of that letter. Each element tells the reader (and the browser) what role that section plays.

**⚡ Intermediate: Why it matters beyond visuals**

Styling can make any element look like anything. A  can look like a heading, a  can look like a button. But the browser, search engines, and assistive technologies rely on the actual element name — not its appearance — to understand content structure.

Semantic HTML is about information architecture, not visual presentation.

## 1.2 Who Reads Your HTML?

When you write HTML, four audiences read it:

| Audience | How They Use Your HTML |
|---|---|
| `Browsers` | Parse the DOM, apply default styles based on element type, handle interaction |
| `Screen Readers` | Announce elements by role — 'heading level 1', 'navigation', 'button'. Rely entirely on semantic markup. |
| `Search Engines` | Crawl your page and weight content based on element importance. <h1> signals 'primary topic'. |
| `Other Developers` | Read your code to understand the page structure. Semantic HTML is self-documenting. |

## 1.3 Semantic vs Non-Semantic Elements

| Non-Semantic (generic) | Semantic (meaningful) |
|---|---|
| `<div>` | <header>, <main>, <footer>, <section>, <article>, <nav>, <aside> |
| `<span>` | , , <mark>, <time>, <abbr>, <cite>, <q> |
| `<b>` |   (important, not just bold) |
| `<i>` |   (emphasis, not just italic) |

**div and span are still useful**

 and  are not wrong — they are the right choice when you need a container purely for CSS or JavaScript purposes and no semantic element fits.

The mistake is using  for EVERYTHING when semantic alternatives exist.

### 1.4 Benefits of Semantic HTML

1. Accessibility — screen readers navigate by landmarks (header, nav, main, footer). Users can jump straight to the main content.
2. SEO — search engines understand page hierarchy. Content in <h1>, <article>, and <main> is weighted higher.
3. Maintainability — code is self-documenting. A <nav> is immediately understood; a  requires reading the class name.
4. Browser defaults — semantic elements come with useful default behaviour (e.g. <button> is keyboard-focusable by default).
5. Future-proofing — HTML standards continue to evolve around semantic meaning.

## 02 Page Structure Elements

# Page Structure Elements

HTML5 introduced a set of landmark elements that define the major sections of a page. These replace the old pattern of using ,  etc.

### 2.1 <header>

The `<header>` element represents introductory content at the top of a page or section. It typically contains the site logo, page title, navigation, and tagline.

```
page-structure.html
<header>
  <img src="logo.png" alt="TS Academy logo">
  <h1>TS Academy</h1>
  <p>Learn to code, build the future</p>
  <nav>
    <!-- navigation goes here -->
  </nav>
</header>
```

**Key Point**

<header> can appear multiple times on a page — once at the page level, and once inside each <article> or <section> to introduce that specific piece of content.

**⚡ Intermediate: <header> vs <head>**

<head> — inside the HTML document, contains metadata (title, meta tags, CSS links). Not visible on the page.

<header> — inside <body>, visible section at the top of the page or a section. These are completely different elements.

## 2.2 <nav>

The `<nav>` element wraps a set of navigation links — the main site menu, a table of contents, breadcrumbs, or pagination.

page-structure.html

```
<nav>
  <ul>
    <li><a href='/'>Home</a></li>
    <li><a href='/courses'>Courses</a></li>
    <li><a href='/about'>About</a></li>
    <li><a href='/contact'>Contact</a></li>
  </ul>
</nav>
```

**Not every list of links needs <nav>**

Use <nav> only for major navigation blocks. A list of social media icons in the footer, or a set of tag links, does not need to be a <nav>.

Screen readers announce the <nav> landmark to users, so reserve it for meaningful navigation.

## 2.3 <main>

The `<main>` element contains the dominant, unique content of the page — the content that is not repeated across pages (unlike headers and footers).

page-structure.html

```
<!-- There should only be ONE <main> per page -->
<main>
  <h1>Introduction to CSS</h1>
  <p>CSS (Cascading Style Sheets) controls how HTML looks...</p>
  <!-- all the main page content goes here -->
</main>
```

**Beginner: One main per page**

Think of <main> as 'the reason this page exists'. There is exactly one <main> on any page. A screen reader user can press a keyboard shortcut to jump straight to the <main> content, skipping the navigation entirely — this is why it matters.

**⚡ Intermediate: Hidden main elements in SPAs**

If you build a single-page application with multiple views, only one <main> should be visible at a time. Hide inactive mains with the hidden attribute or CSS display:none — do not have multiple visible <main> elements on one page.

## 2.4 <article>

An `<article>` is a self-contained, independently distributable piece of content. If you could take it out of the page and it would still make complete sense on its own, it is an article.

```
blog.html
<!-- Blog post — makes sense on its own -->
<article>
  <header>
    <h2>5 Tips for Better HTML</h2>
    <time datetime="2026-03-18">March 18, 2026</time>
    <p>By <a href="/authors/amara">Amara Okafor</a></p>
  </header>
  <p>Good HTML structure starts with choosing the right elements...</p>
  <footer>
    <p>Tags: HTML, Best Practices</p>
  </footer>
</article>
```

Good candidates for <article>:

- A blog post or news article
- A product listing in a shop
- A user comment or forum post
- A widget or gadget that could be syndicated

## 2.5 <section>

A `<section>` groups thematically related content within a larger document. It should always have a heading.

```
page-structure.html
<main>
  <section>
    <h2>Module 1: HTML Basics</h2>
    <p>Introduction to HTML tags, structure, and syntax...</p>
  </section>
  <section>
    <h2>Module 2: HTML Fundamentals</h2>
    <p>Tables, forms, multimedia, and semantic HTML...</p>
  </section>
  <section>
    <h2>Module 3: CSS Styling</h2>
    <p>Selectors, properties, values, and layout...</p>
  </section>
</main>
```

**article vs section — when to use each**

Use <article> when the content could stand alone (blog post, product card, comment).

Use <section> when the content is part of a larger whole and needs grouping (chapters of a tutorial, tabs in a dashboard).

Rule of thumb: Could you post it on another website and it would make sense? → <article>. Is it just a chunk of the current page? → <section>.

## 2.6 <aside>

An `<aside>` contains content that is tangentially related to the main content — it could be removed without breaking the main content's meaning.

```
page-structure.html
<main>
  <article>
    <h2>Introduction to HTML Tables</h2>
    <p>HTML tables organise data into rows and columns...</p>
  </article>
  <aside>
    <h3>Did You Know?</h3>
    <p>Tables were originally used for page layout in the 1990s.</p>
    <p>Today, CSS Grid handles layout — tables are for data only.</p>
  </aside>
</main>
```

Common uses of <aside>:

- Pull quotes or highlighted facts
- Sidebars with related links
- Author biography in an article
- Advertisements (content unrelated to the page topic)

## 2.7 <footer>

A `<footer>` contains footer information for its nearest ancestor — usually copyright, links, contact info, or last-updated date. Like `<header>`, it can appear at both page level and inside individual articles.

page-structure.html

```
<footer>
  <p>&copy; 2026 TS Academy. All rights reserved.</p>
  <nav>
    <a href='/privacy'>Privacy Policy</a>
    <a href='/terms'>Terms of Use</a>
  </nav>
  <address>
    Contact: <a href='mailto:hello@tsacademy.ng'>hello@tsacademy.ng</a>
  </address>
</footer>
```

## 2.8 Complete Page Structure Example

complete-page.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TS Academy — HTML Course</title>
</head>
<body>
  <header>
    <img src="logo.png" alt="TS Academy">
    <nav>
      <ul>
        <li><a href='/'>Home</a></li>
        <li><a href='/courses'>Courses</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <section>
      <h1>HTML Fundamentals</h1>
      <p>Welcome to Module 2. In this module you will learn...</p>
    </section>
    <section>
      <h2>Course Modules</h2>
      <article>
        <h3>Tables</h3>
        <p>Learn to structure data with HTML tables.</p>
      </article>
      <article>
        <h3>Forms</h3>
        <p>Collect user input with HTML forms.</p>
      </article>
```

```
    </section>
    <aside>
      <h2>Quick Tip</h2>
      <p>Use the Emmet shortcut ! + Tab in VSCode to generate this
structure instantly.</p>
    </aside>
  </main>
  <footer>
    <p>&copy; 2026 TS Academy</p>
  </footer>
</body>
</html>
```

## 2.9 Landmark Elements Quick Reference

| Element | Role | Key Rule |
|---|---|---|
| `<header>` | Introductory content for page or section | Can appear multiple times — once per page, article, section |
| `<nav>` | Major navigation links | For significant navigation only — not every link list |
| `<main>` | Primary unique content of the page | Only ONE <main> visible per page |
| `<article>` | Self-contained, independently distributable content | Must make sense if taken out of context |
| `<section>` | Thematic grouping within a larger document | Always give it a heading |
| `<aside>` | Tangentially related content | Removing it should not break the main content |
| `<footer>` | Footer info for page or section | Can appear multiple times like <header> |

**Practice Task — Page Structure (VSCode)**

Create a file called semantic-page.html in VSCode.

Build a blog-style page with:

1. <header> with site name and <nav> with 3 links
2. <main> with: a <section> (welcome), two <article> elements (each with heading + date + paragraph)
3. <aside> with a 'Related Links' section
4. <footer> with copyright and a contact email (<address>)

Open with Live Server and inspect the structure in DevTools (F12 → Elements).

**03** **Text-Level Semantic Elements**

# Text-Level Semantic Elements

Beyond page structure, HTML provides elements for marking up the meaning of individual words and phrases within your text — called inline semantic elements.

## 3.1  and 

###  — Important Text

Use `<strong>` when text is of strong importance, seriousness, or urgency. Browsers render it bold by default, but the meaning is 'this matters', not just 'make this bold'.

```
text-semantics.html
<p><strong>Warning:</strong> Never share your password with anyone.</p>

<!-- Wrong: styling only, no semantic meaning -->
<p>This text is <strong>larger</strong> than the rest.</p>  <!-- Use CSS -
->
```

###  — Emphasis

Use `<em>` for text that needs spoken emphasis. It renders italic by default.

```
text-semantics.html
<!-- Emphasis changes the meaning of each sentence -->
<p>I <em>never</em> said she stole the money.</p>
<p>I never said <em>she</em> stole the money.</p>
<p>I never said she stole <em>the</em> money.</p>
```

** vs  ·  vs **

 = important |  = stylistically bold, no extra importance

 = spoken emphasis |  = alternative voice, technical term, foreign word

Example: Ingredients: flour, sugar (label — not important)

Example: HTML stands for HyperText Markup Language (technical term)

## 3.2 <mark>, <time>, <abbr>

### <mark> — Highlight

The `<mark>` element highlights text relevant in a specific context — like search results marking the matched keyword.

```
text-semantics.html
<!-- User searched for "semantic" -->
<p>Writing good HTML means writing <mark>semantic</mark> markup.</p>
```

### <time> — Dates and Times

Use `<time>` to mark up dates and times. The `datetime` attribute provides a machine-readable version.

```
text-semantics.html
<time datetime="2026-03-18">March 18, 2026</time>
<time datetime="14:30">2:30 PM</time>
<time datetime="2026-03-18T14:30">March 18, 2026 at 2:30 PM</time>
<time datetime="PT2H30M">2 hours 30 minutes</time>
```

> ⚡ **Intermediate: Why datetime matters**
>
> The datetime attribute uses ISO 8601 format (YYYY-MM-DD). This allows search engines to index event dates, calendar apps to import events, and screen readers to announce dates in the user's preferred format.

### <abbr> — Abbreviations

The `<abbr>` element marks an abbreviation or acronym. The `title` attribute provides the full expansion as a tooltip.

```
text-semantics.html
<p><abbr title="HyperText Markup Language">HTML</abbr> is the
   standard language for web pages.</p>
```

## 3.3 <address>, <blockquote>, <cite>

### <address>

The `<address>` element provides contact information for its nearest article or the page as a whole.

```
text-semantics.html
<footer>
  <address>
    TS Academy, Victoria Island, Lagos, Nigeria.
    <a href='mailto:hello@tsacademy.ng'>hello@tsacademy.ng</a>
```

```
  </address>
</footer>
```

### <blockquote> and <cite>

```
text-semantics.html
<blockquote cite="https://developer.mozilla.org/en-
US/docs/Glossary/Semantics">
  <p>Semantics refers to the meaning of a piece of code.</p>
  <footer>— <cite>MDN Web Docs</cite></footer>
</blockquote>

<p>As the spec says, <q>the p element represents a paragraph</q>.</p>
```

## 3.4 Text-Level Elements Quick Reference

| Element | Meaning / Use | Default Style |
|---|---|---|
| `<strong>` | Strong importance | Bold |
| `<em>` | Spoken emphasis | Italic |
| `<b>` | Stylistic bold, no importance | Bold |
| `<i>` | Alternative voice, technical term | Italic |
| `<mark>` | Highlighted / relevant in current context | Yellow background |
| `<time>` | Date or time — use datetime attribute | None |
| `<abbr>` | Abbreviation — use title for full expansion | Dotted underline |
| `<address>` | Contact information | Italic |
| `<blockquote>` | Long quotation from external source | Indented |
| `<q>` | Short inline quotation | Quotes added |
| `<cite>` | Title of a referenced work | Italic |
| `<code>` | Inline code snippet | Monospace |
| `<pre>` | Preformatted text block | Monospace block |

**Practice Task — Text Semantics (VSCode)**

Create text-semantics.html. Write an article about your favourite technology:

1. <article> with <header> containing title and <time> with datetime
2. At least one  for something important
3. At least one  for spoken emphasis
4. An <abbr> for any acronym (HTML, CSS, API...)

5. A <blockquote> with <cite> for a quote from any website
6. An <address> with your name and a mailto link

Open with Live Server and hover over the <abbr> to see the tooltip.

## 04 Heading Hierarchy

# Heading Hierarchy

HTML provides six heading levels: `<h1>` through `<h6>`. These are not just for size — they create a document outline that screen readers, search engines, and other tools rely on.

### 4.1 The Document Outline

Think of headings like the table of contents of a book: the title is H1, chapters are H2, sections within chapters are H3, and so on.

```
heading-hierarchy.html
<h1>TS Academy — HTML Course</h1>          <!-- One per page -->
  <h2>Module 2: HTML Fundamentals</h2>     <!-- Section headings -->
    <h3>HTML Tables</h3>                   <!-- Sub-sections -->
      <h4>colspan and rowspan</h4>         <!-- Sub-sub-sections -->
    <h3>HTML Forms</h3>
      <h4>Input Types</h4>
  <h2>Module 3: CSS Styling</h2>
    <h3>Selectors</h3>
```

### 4.2 Heading Rules

**Rules for correct heading use**

1. Only ONE <h1> per page — it is the main topic of the entire page.
2. Never skip levels — don't jump from <h2> to <h4>. Go h1 → h2 → h3 in order.
3. Don't choose headings based on size — use CSS to control size.
4. Every <section> and <article> should have a heading as its first child.
5. <h1>–<h6> are for document structure, not decorative text.

**Common Mistake vs Correct Approach**

```
WRONG — skipping levels:
  <h1>My Blog</h1>
  <h3>Latest Post</h3>     <!-- Skipped h2 -->
    <h5>Published date</h5>    <!-- Skipped h4 -->

CORRECT — sequential levels:
  <h1>My Blog</h1>
  <h2>Latest Post</h2>
    <h3>Published date</h3>
```

### 4.3 Checking Your Heading Structure

6. Open DevTools (F12)
7. Go to Elements tab → Ctrl+F → search for h1, h2 etc.
8. Or install the HeadingsMap browser extension for a full outline view

**VSCode Tip**

Install the 'HTML Hint' extension in VSCode. It will flag heading hierarchy issues (like skipped levels) directly in the editor with red underlines.

## 05 ARIA & Accessibility Basics

# ARIA & Accessibility Basics

Semantic HTML is the foundation of web accessibility. ARIA (Accessible Rich Internet Applications) is a set of attributes that extend the accessibility information of HTML — but it should only be used when native semantic HTML is not enough.

### 5.1 The First Rule of ARIA

**First Rule of ARIA**

If you can use a native HTML element with the semantics you need, use it — don't add an ARIA role to a generic element.

WRONG: Submit
RIGHT: <button type='submit'>Submit</button>

Native elements have built-in keyboard interaction, focus management, and accessibility for free.

## 5.2 aria-label and aria-labelledby

Use `aria-label` to provide an accessible name when there is no visible text label.

```
aria-examples.html
<!-- Icon button with no visible text -->
<button aria-label="Close dialog">
  <svg><!-- X icon --></svg>
</button>

<!-- Two navs — differentiate them -->
<nav aria-label="Main navigation"><!-- primary menu --></nav>
<nav aria-label="Footer navigation"><!-- footer links --></nav>

<!-- aria-labelledby points to an element's id -->
<section aria-labelledby="services-heading">
  <h2 id="services-heading">Our Services</h2>
</section>
```

## 5.3 aria-hidden

Use `aria-hidden='true'` to remove purely decorative elements from the accessibility tree.

```
aria-examples.html
<!-- Decorative icon — screen reader skips it -->
<p><span aria-hidden="true">□</span> View course materials</p>

<!-- Decorative divider -->
<hr aria-hidden="true">
```

## 5.4 Accessible Landmarks Checklist

Every page should have these accessible landmarks:

9. One `<header>` — site header
10. At least one `<nav>` — main navigation
11. One `<main>` — primary content

12. One `<footer>` — site footer
13. Any `<aside>` elements labelled with aria-label

> ⚡ **Intermediate: Testing with a screen reader**
>
> Test with a free screen reader:
> • NVDA (Windows, free): nvaccess.org/download
> • VoiceOver (Mac): Cmd + F5 to enable
> • ChromeVox: free Chrome Web Store extension
>
> Press H to jump between headings and D to jump between landmarks.

## 06 Common Mistakes & How to Fix Them

# Common Mistakes & How to Fix Them

### Mistake 1 — Div Soup

```
mistakes.html
<!-- WRONG: everything is a div -->
<div class="page-header">
  <div class="site-nav">
    <div class="nav-item"><a href="/">Home</a></div>
  </div>
</div>

<!-- RIGHT: semantic elements -->
<header>
  <nav>
    <ul><li><a href='/'>Home</a></li></ul>
  </nav>
</header>
```

### Mistake 2 — Headings for Styling

```
mistakes.html
<!-- WRONG: using <h3> because it looks the right size -->
<p>Published by:</p>
<h3>Amara Okafor</h3>   <!-- This is NOT a heading -->

<!-- RIGHT: use CSS for visual size -->
<p>Published by: <strong>Amara Okafor</strong></p>
```

## Mistake 3 — Missing Alt Text

```
mistakes.html
<!-- WRONG: no alt -->
<img src="infographic.png">

<!-- WRONG: uninformative alt -->
<img src="infographic.png" alt="image">

<!-- RIGHT: descriptive -->
<img src="infographic.png" alt="Infographic showing 5 benefits of semantic
HTML">

<!-- RIGHT: decorative image gets empty alt -->
<img src="decorative-wave.png" alt="">
```

## Mistake 4 — Non-Semantic Interactive Elements

```
mistakes.html
<!-- WRONG: div acting as button -->
<div class="btn" onclick="submitForm()">Submit</div>
<!-- Not keyboard-focusable, not announced as button -->

<!-- RIGHT: real button -->
<button type="submit">Submit</button>
<!-- Keyboard-focusable, Enter/Space work automatically -->
```

## Mistake 5 — Section Without a Heading

```
mistakes.html
<!-- WRONG: section without a heading -->
<section>
  <p>Some random paragraph text.</p>
</section>
<!-- A <section> without a heading is just a confusing <div> -->

<!-- RIGHT: either add a heading or use <div> -->
<section>
  <h2>About Our Academy</h2>
  <p>TS Academy was founded in 2020...</p>
</section>
```

07 **Full Semantic Page Example**

# Full Semantic Page Example

This example brings together all the semantic elements from this module into a complete, accessible blog article page:

full-semantic-page.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>5 Tips for Better HTML — TS Academy Blog</title>
</head>
<body>

  <header>
    <a href="/"><img src="logo.png" alt="TS Academy"></a>
    <nav aria-label="Main navigation">
      <ul>
        <li><a href='/'>Home</a></li>
        <li><a href='/courses'>Courses</a></li>
        <li><a href='/blog'>Blog</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <header>
        <h1>5 Tips for Writing Better HTML</h1>
        <address><a href='/authors/amara'>Amara Okafor</a></address>
        <time datetime="2026-03-18">March 18, 2026</time>
      </header>

      <section>
        <h2>Introduction</h2>
        <p>Good <abbr title='HyperText Markup Language'>HTML</abbr>
           is not just about making things look right in a browser.
           <strong>Semantic HTML makes your pages more accessible,
           easier to maintain, and better ranked by search
engines.</strong>
        </p>
      </section>

      <section>
        <h2>The 5 Tips</h2>
```

```
        <section>
          <h3>Tip 1: Use landmark elements</h3>
          <p>Replace generic <em>divs</em> with header, main,
             nav, article, section, aside, and footer.</p>
        </section>
        <section>
          <h3>Tip 2: One H1 per page</h3>
          <p>Your H1 is the main topic. Keep it unique and
descriptive.</p>
        </section>
        <section>
          <h3>Tip 3: Write meaningful alt text</h3>
          <figure>
            <img src="alt-text-example.png"
                 alt="Screenshot comparing good and bad alt text
examples">
            <figcaption>Figure 1: Good vs bad alt text</figcaption>
          </figure>
        </section>
      </section>

      <blockquote cite="https://webaim.org">
        <p>The power of the Web is in its universality.</p>
        <footer>— <cite>Tim Berners-Lee</cite></footer>
      </blockquote>

      <footer>
        <p>Tags: HTML, Semantic HTML, Accessibility</p>
      </footer>
    </article>

    <aside aria-label="Related articles">
      <h2>Related Articles</h2>
      <ul>
        <li><a href='/blog/html-forms'>Building Accessible Forms</a></li>
        <li><a href='/blog/css-layout'>CSS Flexbox vs Grid</a></li>
      </ul>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 TS Academy. All rights reserved.</p>
    <nav aria-label='Footer navigation'>
      <a href='/privacy'>Privacy</a>
      <a href='/terms'>Terms</a>
    </nav>
    <address>
      <a href='mailto:hello@tsacademy.ng'>hello@tsacademy.ng</a>
    </address>
  </footer>
</body>
</html>
```

∑ **Summary & Quick Reference**

# Summary & Quick Reference

## Page Structure Elements

| Element | Purpose |
|---|---|
| `<header>` | Site or section header — logo, title, nav |
| `<nav>` | Major navigation links |
| `<main>` | Primary unique content — one per page |
| `<article>` | Self-contained, independently shareable content |
| `<section>` | Thematic grouping — always needs a heading |
| `<aside>` | Tangentially related content — sidebar, callout |
| `<footer>` | Footer info — copyright, contact, links |

## Text-Level Semantics

| Element | Purpose |
|---|---|
| `<strong>` | Strong importance (renders bold) |
| `<em>` | Spoken emphasis (renders italic) |
| `<mark>` | Highlighted / search match |
| `<time>` | Date or time — with datetime attribute |
| `<abbr>` | Abbreviation — with title attribute |
| `<address>` | Contact information |
| `<blockquote>` | Long external quotation |
| `<q>` | Short inline quotation |
| `<cite>` | Title of a referenced work |
| `<figure>` | Self-contained media block |
| `<figcaption>` | Caption for a <figure> |

| Element | Purpose |
|---|---|
| `<code>` | Inline code |
| `<pre>` | Preformatted / code block |

## Key Rules to Remember

14. Only ONE `<main>` and ONE `<h1>` per page.
15. Never skip heading levels — h1 → h2 → h3 in order.
16. Every `<section>` and `<article>` should have a heading.
17. Use semantic elements before reaching for ARIA.
18. `<div>` and `<span>` are for CSS/JS hooks — not for content meaning.
19. Alt text: descriptive for informative images, `alt=""` (empty) for decorative ones.

### Final Task — Semantic HTML Audit (VSCode)

Take any HTML file you have already written and:
1. Find every  — is there a semantic element that fits better?
2. Check for exactly one <h1> and sequential heading levels.
3. Ensure every <img> has a meaningful alt attribute.
4. Wrap the main content area in <main>.
5. Wrap navigation links in <nav>.
6. Add <header> and <footer> where needed.

Open with Live Server and press F12 → Elements to inspect the semantic structure.
Bonus: Run your page through the W3C Validator (validator.w3.org).

### Further Resources

MDN Semantic HTML guide: https://developer.mozilla.org/en-US/docs/Glossary/Semantics#semantics_in_html
W3C HTML Validator: https://validator.w3.org
WebAIM accessibility checker: https://wave.webaim.org
HeadingsMap browser extension: search 'HeadingsMap' in your browser store
ARIA Authoring Practices Guide: https://www.w3.org/WAI/ARIA/apg